

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

Направление 11.03.02 «Инфокоммуникационные технологии и системы связи»

Образовательная программа «Программирование в инфокоммуникационных системах»

Дисциплина «Web-программирование»

Преподаватель:
Богатыренков С.С.

Выполнил:
студент группы К3321
Стафеев И.А.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

	Стр.
ВВЕДЕНИЕ	3
1 Выполнение работы.....	4
ЗАКЛЮЧЕНИЕ	8

ВВЕДЕНИЕ

Цель работы: разработка и развертывание веб-приложения на основе фреймворка Express, обеспечивающего обработку HTTP-запросов, работу с параметрами URL и телом запроса, а также предоставление набора маршрутов для выполнения различных задач.

Для достижения поставленной цели были сформулированы следующие задачи:

- разработать веб-приложение с использованием фреймворка Express и модульной структуры файлов `app.js` и `index.js`;
- реализовать поддержку CORS, включая разрешение всех методов HTTP и необходимых заголовков;
- реализовать маршрут `/login/`, возвращающий логин пользователя в системе LMS;
- разработать маршрут `/code/`, обеспечивающий возврат исходного кода файла приложения средствами потокового чтения файлов;
- реализовать маршрут `/sha1/{input}/`, выполняющий вычисление SHA1-хэша от переданной строки;
- разработать маршрут `/req/`, обеспечивающий получение содержимого удаленного интернет-ресурса по указанному URL-адресу с поддержкой методов GET и POST;
- реализовать обработку всех остальных запросов с возвратом логина пользователя;
- развернуть приложение на удаленном сервере с поддержкой HTTPS и обеспечить его доступность через интернет.

1 Выполнение работы

В файле `index.js` (рисунок 1.1) сначала выполняется импорт необходимых модулей - Express, middleware `body-parser`, функции `createReadStream` из модуля `fs`, стандартных модулей `crypto` и `http`. После этого из файла `app.js` импортируется функция инициализации приложения, которой передаются все нужные параметры. Этот экземпляр Express-приложения запускается на порту, который предоставляется средой выполнения.

```
JS index.js > ...
1  import express from 'express';
2  import bodyParser from 'body-parser';
3  import { createReadStream } from 'fs';
4  import crypto from 'crypto';
5  import http from 'http';
6  import appSrc from './app.js';
7
8  let app = appSrc(express, bodyParser, createReadStream, crypto, http);
9  app.listen(process.env.PORT);
```

Рисунок 1.1 — Код файла `index.js`

Код файла `app.js` разбит на две картинки. На первой (рисунок 1.2) создается экземпляр Express-приложения и подключается middleware для обработки данных тела запроса в форматах `application/x-www-form-urlencoded` и `JSON`. Далее идет middleware для поддержки CORS, устанавливающий необходимые HTTP-заголовки. Здесь же реализована политика `trailing slashes`, работает она так: если путь запроса не оканчивается символом `/`, выполняется перенаправление на корректный URL с использованием кода 301.

```

JS app.js > default
1  export default (express, bodyParser, createReadStream, crypto, http) => {
2      let app = express();
3      app.use(bodyParser.urlencoded({ extended: false }));
4      app.use(bodyParser.json());
5      app.use((req, res, next) => {
6          res.setHeader("Access-Control-Allow-Origin", "*");
7          res.setHeader("Access-Control-Allow-Methods", "*");
8          res.setHeader(
9              "Access-Control-Allow-Headers",
10             "Access-Control-Allow-Headers, Content-Type, x-author, ngrok-skip-browser-warning"
11         );
12         if (!req.path.endsWith("/")) {
13             let query = req.url.includes("?") ? req.url.substring(req.url.indexOf("?")) : "";
14             return res.redirect(301, req.path + "/" + query);
15         }
16         next();
17     });
18

```

Рисунок 1.2 — Создание экземпляра приложения в `app.js`

Обработка маршрутов показана на рисунке 1.3. Маршрут `/login/` возвращает логин пользователя, маршрут `/code/` выводит исходный код файла `app.js`, для чего используется потоковое чтение файла через `createReadStream`. Маршрут `/sha1/{input}/` вычисляет SHA1-хэш от строки, переданной в параметре URL, для этого используется модуль `crypto`.

Следом реализован маршрут `/req/`, поддерживающий методы GET и POST. Маршрут получает URL-адрес из query-параметра или тела запроса, после чего выполняет HTTP-запрос к указанному ресурсу с помощью метода `http.get`, а содержимое возвращается в качестве ответа. В самом конце для обработки всех остальных запросов идет маршрут `app.all("*")`, возвращающий логин.

```

18
19 app.get("/login/", (req, res) => {res.send("staffeev409626");});
20 app.get("/code/", (req, res) => {createReadStream(import.meta.url.substring(7)).pipe(res);});
21 app.get("/sha1:input/", (req, res) => {
22     let hash = crypto.createHash("sha1").update(req.params.input).digest("hex");
23     res.send(hash);
24 });
25
26 let reqHandler = (req, res) => {
27     let addr = (req.method === "POST") ? req.body.addr : req.query.addr;
28
29     http.get(addr, response => {
30         let data = "";
31         response.on("data", chunk => {data += chunk;});
32         response.on("end", () => {res.send(data);});
33     });
34 };
35 app.get("/req/", reqHandler);
36 app.post("/req/", reqHandler);
37
38 app.all("*", (req, res) => {res.send("staffeev409626");});
39 return app;
40 };

```

Рисунок 1.3 — Обработка маршрутов в `app.js`

Для проверки работоспособности были отправлены все необходимые запросы к веб-сервису (размещен с помощью Render), результат показан на рисунке 1.4. Как видно, все маршруты отрабатывают корректно.

```
Администратор: Командная строка

C:\Windows\System32>curl "https://web26-lab3.onrender.com/login/"
staffeev409626
C:\Windows\System32>curl "https://web26-lab3.onrender.com/sha1/hello/"
aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d
C:\Windows\System32>curl "https://web26-lab3.onrender.com/req/?addr=http://node-server.online/r/assets/week.txt"
5
C:\Windows\System32>curl "https://web26-lab3.onrender.com/req/" -d "addr=http://node-server.online/r/assets/week.txt"
5
C:\Windows\System32>curl "https://web26-lab3.onrender.com/random_route/"
staffeev409626
C:\Windows\System32>curl "https://web26-lab3.onrender.com/code/"
export default (express, bodyParser, createReadStream, crypto, http) => {
  let app = express();
  app.use(bodyParser.urlencoded({ extended: false }));
  app.use(bodyParser.json());
  app.use((req, res, next) => {
    res.setHeader("Access-Control-Allow-Origin", "*");
    res.setHeader("Access-Control-Allow-Methods", "*");
    res.setHeader(
      "Access-Control-Allow-Headers",
      "Access-Control-Allow-Headers, Content-Type, x-author, ngrok-skip-browser-warning"
    );
    if (!req.path.endsWith("/")) {
      let query = req.url.includes("?") ? req.url.substring(req.url.indexOf("?")) : "";
      return res.redirect(301, req.path + "/" + query);
    }
    next();
  });

  app.get("/login/", (req, res) => {res.send("staffeev409626");});
  app.get("/code/", (req, res) => {createReadStream(import.meta.url.substring(7)).pipe(res);});
  app.get("/sha1:input/", (req, res) => {
    let hash = crypto.createHash("sha1").update(req.params.input).digest("hex");
```

Рисунок 1.4 — Выполнение запросов к веб-сервису

В тестирующей системе веб-сервис тоже отработал корректно (рисунок 1.5).

Задание по ExpressJS

-

100,00

0-100

100,00 %

-

Рисунок 1.5 — Успешное выполнение в тестирующей системе

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы было разработано и развернуто веб-приложение на основе Express. Были реализованы маршруты для обработки HTTP-запросов, вычисления SHA1-хэша, получения содержимого удаленного ресурса и возврата исходного кода приложения. Также была обеспечена поддержка CORS и политики trailing slashes. В результате работы были получены практические навыки разработки серверных приложений на Node.js и их развертывания с использованием HTTPS. Цель работы достигнута.